

Large Language Models and Alignment ——Homework1

张硕 2401213265

2025 年 6 月 16 日

1 Tokenization

1.1 实现 BPE, 训练 Tokenizer

1.1.1 简述 BPE 算法和基于 BPE 算法训练 LLM tokenizer 的流程

BPE (Byte Pair Encoding) 算法是一种**文本压缩算法**, 最早由 Philip Gage 于 1994 年提出, 用于数据压缩, 后来被广泛应用于自然语言处理 (NLP) 任务, 其基本思想是通过“合并”文本中的常见字符对来减少表示的字符数量, 从而简化文本的表示, 具体过程如下: 算法通过迭代地统计文本中最常见的字符对, 并将其合并为新的符号, 从而逐步减少词汇表的大小。BPE 起始时将文本拆解为字符级单位, 接着通过多轮合并频繁出现的字符对, 最终得到一个包含子词 (subwords) 的词汇表。通过这个方法, 原本的多个字符逐渐被合并成更大、更有意义的单位, 文本变得更简短。

在训练一个大型语言模型时, 我们需要一个**分词器** (tokenizer), 它的任务是将文本拆分成模型可以理解的基本单元 “tokens”。BPE 算法是实现这种分词器的一种方法。基于 BPE 训练一个 tokenizer 的过程如下:

- **输入文本准备:** 首先, 我们收集大量的文本数据, 作为训练 tokenizer 的原料。这里的原料可以来自新闻、书籍、网络文章等多种来源。
- **分割成字符:** BPE 的训练从将每个单词拆解成最小的单位 (通常是单个字符) 开始。比如, “hello” 会被拆成 *h, e, l, l, o* 这些字符。
- **统计字符对频率:** 接着, 我们会统计这些字符对的出现频率。例如, “h” 与 “e” 组合在一起出现了多少次, “e” 与 “l” 组合出现了多少次, 等等。
- **合并最常见的字符对:** BPE 会根据频率选择出现最频繁的字符对进行合并。例如, 如果字符对 “l” 和 “o” 是最常见的, 它们就会被合并成一个新的 token “lo”。然后更新文本, 使用新的 “lo” 代替原来的字符对。
- **重复合并:** 这一过程会不断重复, 每次都合并频率最高的字符对。直到达到预定的合并次数或没有更多的字符对可以合并为止。
- **最终 tokenizer:** 经过多轮合并后, 我们最终会得到一个包含多个子词 (subwords) 作为 token 的词汇表。这个词汇表中的 token 可以是完整的词, 也可以是词的一部分 (例如词根、前缀或后缀等)。

这样训练出来的 tokenizer 就能高效地将任何输入文本转化为 token 序列, 供语言模型使用。

1.1.2 实现一个基于 BPE 算法的 tokenizer

具体实现见路径——hw1-code/bpe/bpe.ipynb，其中的主要函数如下 (chatgpt 4o 总结后修改)：

1. 初始化阶段 (`__init__`):

- `vocab`: 词汇表，用于存储每个词（由字符或子词组成的元组）及其频率。
- `merges`: 用于记录在训练过程中按照频率合并的字符（或子词）对，保留其顺序。
- `token2id` 与 `id2token`: 分别表示子词到整数 ID 的映射与其反向映射，用于编码与解码。

2. 训练阶段 (`train`):

- 将输入文本按空格分词，将每个词分解为字符序列并在末尾添加特殊标记 `</w>` 表示词边界。
- 构建初始词汇表（字符级），并计算所有相邻字符对的频率。
- 在每轮迭代中，选出频率最高的字符对，进行合并，更新词表。
- 重复上述过程，直到合并次数达到设定的词汇表大小。
- 最后统计词表中所有子词，构建 `token2id` 和 `id2token`。

3. 编码阶段 (`encode`):

- 将输入文本分词，每个词被拆解为字符序列，并添加词边界标记 `</w>`。
- 按照训练时记录的合并顺序，依次合并匹配的字符对，直至不能继续合并。
- 每个词最终被表示为多个子词单元，将这些子词映射为对应的 ID，形成编码结果。
- 注意：若采用按空格拆词（如原实现），则可能无法保留原始文本中的所有空白与格式信息。

4. 解码阶段 (`decode`):

- 接收一组 ID，使用 `id2token` 还原为对应的子词序列。
- 将子词连接成字符串，并移除结尾标记 `</w>`。
- 最后，删除合并子词之间的空格，以恢复原始文本（但可能无法恢复精确格式，具体取决于 `encode` 实现）。

1.1.3 用《北京大学研究生手册（2023 版）》训练我的 tokenizer

具体实现见路径——hw1-code.ipynb，每一步骤配备有详细的注释

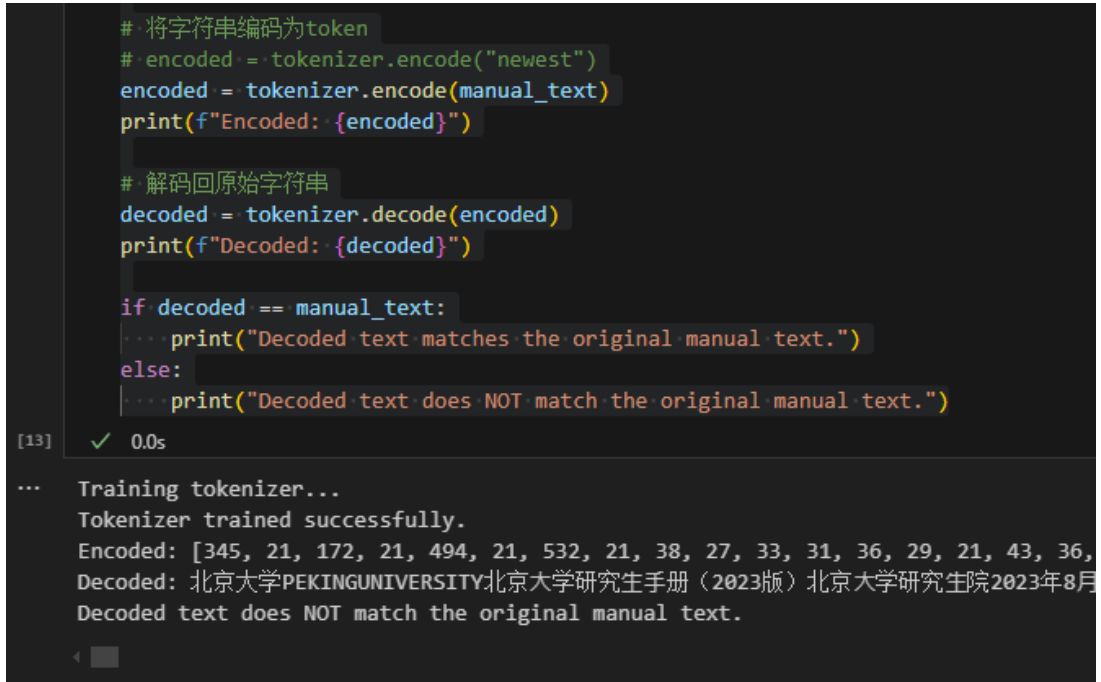
1.1.4 检查编码解码前后 `manual.txt` 文件是否相同

根据算法运行结果来看，编码解码后的 `manual` 文件与原始文件并不相同，但我们人工检查其中内容发现没有明显差异，从原始代码检查分析来看，可以找到原因：即在使用 `split()` 函数在处理文本时，会删除所有空白字符之间的重要信息，导致原始格式丢失。具体而言，它会将多个空格合并为一个空格，将换行符 `\n` 当作分隔符而被忽略，制表符 `\t` 也会被无视，另外文本首尾的空白字符还会被 `strip()` 自动去除。这会使得通过编码再解码得到的文本无法精确还原原文格式。

因此解决方案也比较清晰：把 `split()` 替换成逐字符处理的方式即可解决问题

Listing 1: 逐字符处理

```
def encode(self, text):
    tokens = []
    for token in re.findall(r'\S+|\s', text):
        tokens.extend(self.encode_word(token))
    return tokens
```



```
# 将字符串编码为token
# encoded = tokenizer.encode("newest")
encoded = tokenizer.encode(manual_text)
print(f"Encoded: {encoded}")

# 解码回原始字符串
decoded = tokenizer.decode(encoded)
print(f"Decoded: {decoded}")

if decoded == manual_text:
    print("Decoded text matches the original manual text.")
else:
    print("Decoded text does NOT match the original manual text.")
```

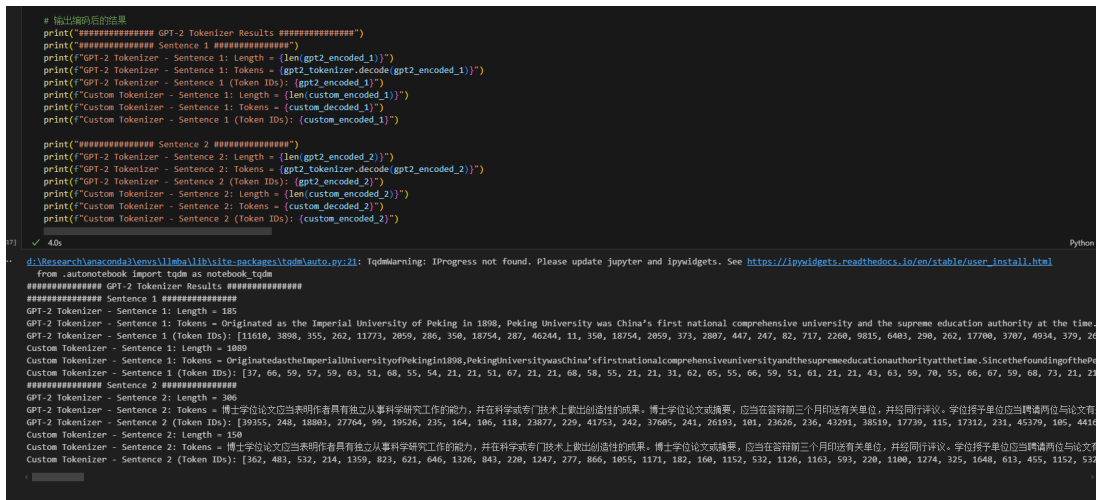
[13] ✓ 0.0s

```
... Training tokenizer...
Tokenizer trained successfully.
Encoded: [345, 21, 172, 21, 494, 21, 532, 21, 38, 27, 33, 31, 36, 29, 21, 43, 36,
Decoded: 北京大学PEKINGUNIVERSITY北京大学研究生手册（2023版）北京大学研究生院2023年8月
Decoded text does NOT match the original manual text.
```

图 1: 运行结果展示

1.1.5 使用 transformers 中的 tokenizer 并对比

具体代码实现见路径 hw1-code/bpe/bpe.py, gpt2 与我自己的 tokenizer 的具体输出如下所示



```
# 输出编码后的结果
print("##### GPT-2 Tokenizer Results #####")
print("##### Sentence 1 #####")
print(f"GPT-2 Tokenizer - Sentence 1: Length = {len(gpt2_encoded_1)}")
print(f"GPT-2 Tokenizer - Sentence 1: Tokens = {gpt2_tokenizer.decode(gpt2_encoded_1)}")
print(f"GPT-2 Tokenizer - Sentence 1 (Token IDs): {gpt2_encoded_1}")
print(f"Custom Tokenizer - Sentence 1: Length = {len(custom_encoded_1)}")
print(f"Custom Tokenizer - Sentence 1: Tokens = {custom_decoded_1}")
print(f"Custom Tokenizer - Sentence 1 (Token IDs): {custom_encoded_1}")

print("##### Sentence 2 #####")
print(f"GPT-2 Tokenizer - Sentence 2: Length = {len(gpt2_encoded_2)}")
print(f"GPT-2 Tokenizer - Sentence 2: Tokens = {gpt2_tokenizer.decode(gpt2_encoded_2)}")
print(f"GPT-2 Tokenizer - Sentence 2 (Token IDs): {gpt2_encoded_2}")
print(f"Custom Tokenizer - Sentence 2: Length = {len(custom_encoded_2)}")
print(f"Custom Tokenizer - Sentence 2: Tokens = {custom_decoded_2}")
print(f"Custom Tokenizer - Sentence 2 (Token IDs): {custom_encoded_2}")
```

✓ 40s

```
d:\Research\anaconda3\envs\lib\site-packages\tdqm\py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
from ..autonotebook import tqdm as notebook_tqdm

##### GPT-2 Tokenizer Results #####
##### Sentence 1 #####
GPT-2 Tokenizer - Sentence 1: Length = 185
GPT-2 Tokenizer - Sentence 1: Tokens = Originated as the Imperial University of Peking in 1898, Peking University was China's first national comprehensive university and the supreme education authority at the time.
GPT-2 Tokenizer - Sentence 1 (Token IDs): [11610, 3898, 355, 262, 11773, 2059, 286, 350, 18754, 287, 46244, 11, 350, 18754, 2059, 373, 2807, 447, 247, 82, 717, 2260, 9815, 6403, 290, 262, 17700, 3707, 4934, 379, 20
Custom Tokenizer - Sentence 1: Length = 1089
Custom Tokenizer - Sentence 1: Tokens = Originated as the Imperial University of Peking in 1898, Peking University was China's first national comprehensive university and the supreme education authority at the time. Since the founding of the h
Custom Tokenizer - Sentence 1 (Token IDs): [37, 66, 59, 57, 59, 63, 51, 68, 55, 54, 21, 21, 51, 67, 21, 21, 68, 58, 55, 21, 21, 31, 62, 65, 55, 66, 59, 51, 61, 21, 21, 43, 63, 59, 70, 55, 66, 67, 59, 68, 73, 21, 2
##### Sentence 2 #####
GPT-2 Tokenizer - Sentence 2: Length = 386
GPT-2 Tokenizer - Sentence 2: Tokens = 博士学位论文应当表明作者具有独立从事科学研究工作的能力，并在科学或专门技术上做出创造性的成果。博士学位论文摘要，应当在答辩前三个月印送有关单位，并送同行评议。学位授予单位应当聘请两位与论文有
GPT-2 Tokenizer - Sentence 2 (Token IDs): [39355, 248, 18083, 27764, 59, 19526, 235, 164, 106, 118, 23877, 229, 41753, 242, 37005, 241, 26193, 101, 23626, 236, 43291, 38519, 37739, 115, 17312, 231, 45379, 165, 4416
Custom Tokenizer - Sentence 2: Length = 150
Custom Tokenizer - Sentence 2: Tokens = 博士学位论文应当表明作者具有独立从事科学研究工作的能力，并在科学或专门技术上做出创造性的成果。博士学位论文摘要，应当在答辩前三个月印送有关单位，并送同行评议。学位授予单位应当聘请两位与论文有
Custom Tokenizer - Sentence 2 (Token IDs): [362, 483, 532, 214, 1359, 823, 621, 646, 1326, 843, 220, 1247, 277, 866, 1055, 1171, 182, 160, 1152, 532, 1126, 1163, 593, 220, 1100, 1274, 325, 1048, 613, 455, 1152, 532
```

图 2: 修改 TrainingArguments

在对同一句子使用 Hugging Face 的 GPT-2 Tokenizer 和我自己实现的 BPE Tokenizer 编码后，

发现两者在 token 数量和具体内容上都存在显著差异，我个人分析原因主要来自以下几点：GPT-2 使用的是基于大规模语料预训练的字节级别的子词编码，能有效识别高频词或词片段，对英文可以说特别有效，因此在处理英文句子时 token 数量较少、粒度更粗。而你自定义的 Tokenizer 只是从 manual.txt 文件中训练，是基于字符级别训练的 BPE，实现较为简化，主要从字符中学习合并规则，由于训练语料较小，英文无法合并成有效词块，token 极为碎片化，因此在英文句子上编码长度远大于 GPT-2。而在中文句子中，由于按汉字作为最小单元处理，每个字一个 token，反而比分字节处理的 GPT-2 更紧凑，因此 token 数量更少。除此之外，GPT-2 的 tokenizer 还支持标点、空格、换行等特殊 token 的处理，而我这里的实现则忽略了这些格式信息。

我将上述原因分析输入给 gpt-4o，它给我原因分析总结如下：要让自定义 tokenizer 更接近 GPT-2，可尝试引入 byte-level 编码、扩充训练语料，以及改进中文处理策略，使其能泛化出更合理的子词单位。

1.2 回答问题

1.2.1 问题一

答案见文件路径——hw1-code/bpe/unicode.ipynb

1.2.2 问题一

具体代码实现见路径 hw1-code/bpe/bpe.py，gpt2 的 tokenizer 的具体输出如下所示



```
查看字符的 Unicode 编码: ord(char)

print(ord('北')) # 输出: 21271
print(ord('大')) # 输出: 22823

[1]
... 21271
    22823

将 Unicode 编码转换为字符: chr(code)

print(chr(22823)) # 输出: 大
print(chr(27169)) # 输出: 模
print(chr(22411)) # 输出: 型

[2]
... 大
    模
    型
```

图 3: 修改 TrainingArguments

1.2.3 问题二

词汇表 (vocab size) 较大时，每个 token 表达的信息更多，序列更短，能更准确还原原文，但模型参数增多，占用内存大，泛化能力弱。相反，词汇表较小时，模型更轻量，适应性更强，能灵活组合新词，但会导致序列变长，计算开销增加，解码可读性降低。因此，词表大小应根据任务和资源平衡选择。

1.2.4 问题三

LLM 不是基于规则或算法的模型，而是基于统计和模式匹配的语言预测器。这使得它在处理**语义丰富但结构不严谨**的自然语言任务时表现很好，但在**结构严格、要求精确**的字符串处理任务上容易出错。LLM 的输入文本首先会被拆分为 **token** 而不是单个字符，所以当模型反转字符串时，它可能试图在 token 级别理解你，而不是按字符顺序处理。LLM 是基于大规模语言统计学习的，它记忆了很多例子，但并不真正理解操作规则。字符串反转是一个确定性算法任务，需要精准处理每个字符的位置——而 LLM 倾向于猜测合理输出而不是执行精确步骤。如果训练数据中没有大量明确标注“字符串反转”任务的例子，模型可能从未学过这种映射关系。所以它可能会看起来像在反转，但出现错字、遗漏或顺序错误。

1.2.5 问题四

LLM 在非英语语言上的表现较差，并非因语言本身简单或复杂，主要是因为训练数据偏英语、语言结构复杂、token 表示低效、以及模型偏向英文语料等多因素共同作用的结果，导致理解能力和生成质量下降。

1. **训练数据偏向英语**：大多数预训练语料（如 Wikipedia、Books、Common Crawl）主要以英文为主，非英语语言尤其是日语在其中占比很低，导致模型对其学习不足。
2. **语言结构差异大**：英语使用空格分词，单词边界清晰；而日语使用混合文字系统（平假名、片假名和汉字），且无空格分词，语法结构更灵活、上下文依赖更强，增加了建模难度。
3. **Token 表示效率低**：LLM 在处理日语时常将一个词拆成多个低频 token，使得同样信息需要更多 token 表示，降低了表示效率与准确性。
4. **标注数据稀缺**：高质量的指令数据、问答、摘要等监督样本在日语中远少于英文，限制了模型在具体任务中的表现。
5. **模型架构偏向英文语料**：许多模型最初在英语语料上设计和调优，对非英语语言并未做特殊优化，迁移效果有限。
6. **长尾语言效应**：训练过程中高频语言占主导地位，低资源语言（如日语）在训练过程中的“存在感”较弱，模型容易忽略其语义规律。

1.2.6 问题五

大型语言模型（LLM）在简单算术问题上的表现往往不如预期，主要原因在于其本质是语言建模器而非逻辑计算器。模型通过学习大规模文本预测最可能的下一个词，而非执行真实的数学运算；算术任务需要符号级的精确推理和中间状态记忆，例如逐位加法与进位处理，而这类结构化推理超出了 LLM 的原生能力；训练语料中涉及系统性数学计算的样本非常稀少，模型往往只能“模仿答案模式”而非真正理解运算逻辑。

Note：但这个问题目前可以通过调用工具很优雅的解决（如把算数问题转化为 python 程序）

1.2.7 问题六

GPT-2 在编写 Python 代码时面临比预期更多的困难，主要原因在于其模型容量有限，难以捕捉代码所需的精确语法与逻辑结构。GPT-2 并未针对代码生成任务进行专门训练，其预训练目标仅为语言建模，缺乏指令理解能力与语义对齐能力。训练语料中高质量代码数据稀少，模型对编程语言的掌握有限。由于 GPT-2 是纯粹的语言模型，没有指令微调（SFT）、人类反馈微调（如 RLHF）和长上下文建模能力，其在理解函数间引用、模块组织等方面表现不佳。

1.2.8 问题七

大型语言模型 (LLM) 在遇到字符串 `<|endoftext|>` 时会突然中断, 是因为该字符串在模型中被定义为特殊的结束标记 token。在训练过程中, 模型习惯在文本结尾看到该 token, 并学习到其后的内容通常为填充或终止信号。在生成时一旦模型预测出 `<|endoftext|>`, 就会触发停止机制, 立即结束文本生成。如果用户在输入 prompt 中显式包含该标记, 也可能被模型误认为是终止信号, 从而提前中断输出。

1.2.9 问题八

在向大型语言模型 (LLM) 提问诸如 “SolidGoldMagikarp” 之类奇特词语时, 模型可能表现出崩溃、卡顿或输出异常的情况。这是因为该类词在模型使用的 BPE 分词器中被视为极其罕见的独立 token, 几乎未出现在训练语料中, 模型对其语义表示训练不足。当这些稀有 token 出现在输入中时, 模型无法有效地预测其上下文, 可能导致生成行为紊乱, 甚至触发停止或异常终止。这类现象揭示了语言模型在 token 覆盖不均与长尾数据建模方面的局限。

1.2.10 问题九

在与大型语言模型 (LLM) 交互时, 更推荐使用 YAML 而非 JSON 作为结构化数据格式。这是因为 YAML 的语法更接近自然语言, 书写简洁, 容错性强, 便于模型生成; 而 JSON 虽然结构明确, 但语法严格, 易出现引号、逗号、嵌套等格式错误, 导致解析失败。YAML 也更适合嵌入提示词中用于 few-shot 示例或函数参数描述, 能有效降低模型生成结构化数据时的复杂度与出错率。

1.2.11 问题十

尽管大型语言模型 (LLM) 在训练阶段以 “预测下一个 token” 为目标, 具备端到端语言建模的表象, 但在实际使用中并非真正意义上的端到端系统。输入文本需通过不可微、独立训练的分词器 (tokenizer) 预处理, 其结果再输入模型; 为完成具体任务往往依赖手工构造的 prompt 模板、few-shot 示例等中间设计; 输出结果还需经过后处理程序解析与验证, 例如提取 JSON、调用外部函数等。更进一步, 模型的函数调用、工具使用等能力实质上是通过语言生成与外部解释模块协作完成。LLM 更像是 “语言生成核心 + 多层交互结构” 的系统, 而非端到端执行器。

2 LLM Implementation

这部分的作业要求是: 理解 LLM 工作原理, 熟悉以 GPT-2 为代表的 autoregressive LLM/decoder-only transformer 的代码实现和常见优化, 能够自己从零实现 LLM 训练全流程代码。代码包的路径在 `hw1-code/pretrain/gpt2`。

Commits		
main	All users	All time
Commits on Jun 16, 2025		
作业二latex文件	KaileZhu committed 2 hours ago	2b1bf4f
作业一latex文件	KaileZhu committed 2 hours ago	1d9ea59
bonus	KaileZhu committed 2 hours ago	611ebb0
Create text_to_text_dpo.ipynb	KaileZhu committed 2 hours ago	d9fdff82
2401213265 添加 Hellaswag 评估指标与可视化	KaileZhu committed 18 hours ago	952d53d
2401213265 添加验证集划分与 loss 可视化	KaileZhu committed 18 hours ago	a2145a1
2401213265 加入 FlashAttention 替换标准 Attention	KaileZhu committed 18 hours ago	2be4c6f
2401213265 实现梯度累积支持	KaileZhu committed 18 hours ago	1c78cad
lora	KaileZhu committed 18 hours ago	0789461
删除lora	KaileZhu committed 18 hours ago	fc1d16d
Commits on Jun 15, 2025		
2401213265添加学习率调度器	KaileZhu committed yesterday	ca5dec5
2401213265 支持 float16 与 tf32 混合精度训练	KaileZhu committed yesterday	76a0498
Commits on Jun 14, 2025		
2401213265 添加 AdamW 优化器及梯度裁剪	KaileZhu committed 2 days ago	c862908
Commits on Jun 12, 2025		
2401213265 添加交叉熵损失函数	KaileZhu committed 4 days ago	93bc178
Commits on Apr 21, 2025		
2401213265 加载 Tiny Shakespeare 数据并构造批量数据集	KaileZhu committed on Apr 21	9bc53bd
2401213265 添加自动设备检测与切换逻辑	KaileZhu committed on Apr 21	c6b9dbf
2401213265 编写基础训练循环代码	KaileZhu committed on Apr 21	b8e76f2
2401213265 编写 forward 函数, 支持前向推理	KaileZhu committed on Apr 21	f5bcd9b
2301213265 实现 GPT2 模型主体结构 (nn.Module)	KaileZhu committed on Apr 21	44ad7fa
2401213265 初始化项目结构与依赖配置	KaileZhu authored on Apr 21	101c560

图 4: commit 记录

2.1 实验笔记

2.1.1 初始化项目结构与依赖配置

- **实现内容**：创建了基本的项目目录结构，包括 `models`、`data`、`scripts` 等文件夹，同时添加了 `requirements.txt` 和 `.gitignore`。
- **实现效果**：首先是清晰了要求的项目结构，也方便后续维护。

2.1.2 实现 GPT2 模型主体结构 (nn.Module)

- **实现内容**: 使用 PyTorch 搭建了 GPT2 的基本模块, 包括 embedding、multi-head attention、MLP 以及 LayerNorm 等组件。
- **实现效果**: 模型可以被成功实例化了, 虽然结构复杂, 但逐层调试后勉强能跑起来。对 Transformer 的理解也更加清楚了。

2.1.3 编写 forward 函数, 支持前向推理

- **实现内容**: 实现了 forward 函数中的输入处理、mask 应用、多层堆叠并返回 logits。
- **实现效果**: 模型能正常输出 token 的 logits 了, 第一次看到输出虽然很乱, 但总算跑通了前向过程, 很爽。

2.1.4 编写基础训练循环代码

- **实现内容**: 实现了一个基本的训练 loop, 包括前向传播、反向传播和 optimizer.step。
- **实现效果**: loss 会随着 epoch 稳定下降, 过程有点慢了。

2.1.5 添加自动设备检测与切换逻辑

- **实现内容**: 增加了对 CUDA、MPS 和 CPU 的自动检测与适配逻辑。
- **实现效果**: 模型可以自动切换到最快的可用设备, 免去了频繁修改代码的麻烦, 增加了点通用性。

2.1.6 加载 Tiny Shakespeare 数据并构造批量数据集

- **实现内容**: 加载 Tiny Shakespeare 数据集, 并将文本切分成固定长度的输入序列, 构造 batch。
- **实现效果**: 数据可以被顺利输入模型, 避免了每次手动切片的麻烦, 训练也更稳定。

2.1.7 添加交叉熵损失函数

- **实现内容**: 使用 nn.CrossEntropyLoss 作为训练时的损失函数。
- **实现效果**: 模型终于可以开始学习了! loss 有了明显下降, 但是偶尔会波动。

2.1.8 添加 AdamW 优化器及梯度裁剪

- **实现内容**: 配置了 AdamW 优化器, 并添加了梯度裁剪来防止梯度爆炸。
- **实现效果**: 训练稳定性提升, 很少会出现 loss 突然变大的情况。

2.1.9 支持 float16 与 tf32 混合精度训练

- **实现内容**: 在支持的 GPU 上启用了 AMP 和 tf32 加速训练。
- **实现效果**: 训练时间明显缩短, 显存使用也更少了, 第一次用混合精度, 学习到了。

2.1.10 添加学习率调度器 (Cosine Annealing + Warmup)

- **实现内容**: 引入了学习率预热和余弦退火策略。
- **实现效果**: 训练更平滑了, 前期不会一下子 loss 巨降后发散, 后期也能更好地收敛。

2.1.11 实现梯度累积支持

- **实现内容**: 添加了 gradient accumulation 的逻辑, 用于在显存受限时等效增加 batch size。
- **实现效果**: 在 6GB 显存的显卡上也能训练很大的 batch 。

2.1.12 加入 FlashAttention 替换标准 Attention

- **实现内容**: 将标准多头注意力模块替换为 FlashAttention。
- **实现效果**: 训练速度明显提升, 尤其是在序列较长时, 降低了现存的压力。

2.1.13 支持 FineWeb EDU 数据集

- **实现内容**: 将原来的 Tiny Shakespeare 数据换成了 FineWeb EDU 数据集。
- **实现效果**: 生成的文本更加丰富和合理, 语义上更加通顺。

2.1.14 添加验证集划分与 loss 可视化

- **实现内容**: 实现了训练集/验证集划分, 并记录 loss 曲线用 matplotlib 可视化。
- **实现效果**: 可以很清楚地看到模型有没有过拟合或欠拟合, 对调参也有帮助。

2.1.15 集成 Hellaswag 评估与可视化

- **实现内容**: 集成了 Hellaswag 数据集, 加入测试流程, 并绘制评估指标变化曲线。
- **实现效果**: 除了 loss 之外, 有了更贴近实际下游任务的评估方式, 训练过程也更有目标。

2.2 收获

复现 GPT-2 的过程, 我真正感受到从“概念”走向“代码”的跨度有多大。看似简单的架构图和论文描述, 在真正写模型结构时, 每一个细节都成了必须认真对待的问题。尤其是自注意力机制的实现, 最让我惊讶的是, 它竟然可以只靠几个矩阵乘法和 softmax, 就实现了复杂的序列建模能力。这种“简洁而强大”的设计思路给了我很大启发。mask 的实现也让我意识到“看起来是小细节, 其实是关键控制点”。如果不正确使用下三角 mask, 模型训练时就可能“偷看未来”, 这个问题在开始时完全没意识到, 是通过调试和查阅文档才慢慢搞懂的。过程中我踩坑最多的地方, 就是张量形状的管理。在多头注意力里, Q、K、V 的 reshape、transpose 操作稍微不对, 结果就会完全错乱, 有时候甚至 loss 直接变成 NaN。对张量维度的敏感度。哪怕是大家都在用的 GPT-2, 只要自己真的动手实现过一遍, 理解就完全不一样了。每一个 for 循环, 每一个维度, 每一个操作的语义, 都是构成整体模型稳定运行的基石。

3 LoRA Fine-tuning

3.1 完整地运行代码，提交运行记录

在 colab 上完整运行助教老师提供的代码，将带有运行结果的文件保存在/hw1-code/lora/fork “Lora_ipynb” .ipynb 路径下（本地运行时需要安装 pytorch 等相关库），示例代码加载 bloomz-560m 模型及其 tokenizer，给出示例推理函数，并给出了简单示例；之后，colab 上的代码加载了 imdb_review_3000 数据集，并查看了数据的一个样本；随后进行 lora 微调，使用 peft 设定 lora 微调的参数，并用它来包装模型，创建工作区用以保存模型，使用 transformer 库的 TrainingArguments 类定义训练的参数，使用实例化的 Trainer 类中的 train 函数进行训练，训练过程的权重配置等参数保存在 wandb.ai 网站中（需要输入该网站中个人账号对应的 api 来进行训练），使用 colab 上的 gpu 训练速度较快，两分钟左右可以完成微调过程，最后就是保存模型，并再次测试，可以得到更加优质的回答。

3.2 完成首次训练，提交训练 loss 曲线和模型生成结果

根据 chatgpt 的提示，只需要修改 TrainingArguments，就可以得到训练 loss 曲线如下图所示：

设定训练参数

```
[12] import transformers
      from transformers import TrainingArguments, Trainer
      training_args = TrainingArguments(
          output_dir=output_directory,
          auto_find_batch_size=True, # Find a correct batch size that fits the size of Data.
          learning_rate= 3e-2, # Higher learning rate than full fine-tuning.
          num_train_epochs=2,
          report_to="wandb",
          logging_steps=1, # 每步记录一次
          # logging_dir="./logs" # 保留本地日志
          use_cpu=False
      )
```

图 5: 修改 TrainingArguments

得到的训练 loss 曲线如下图所示：

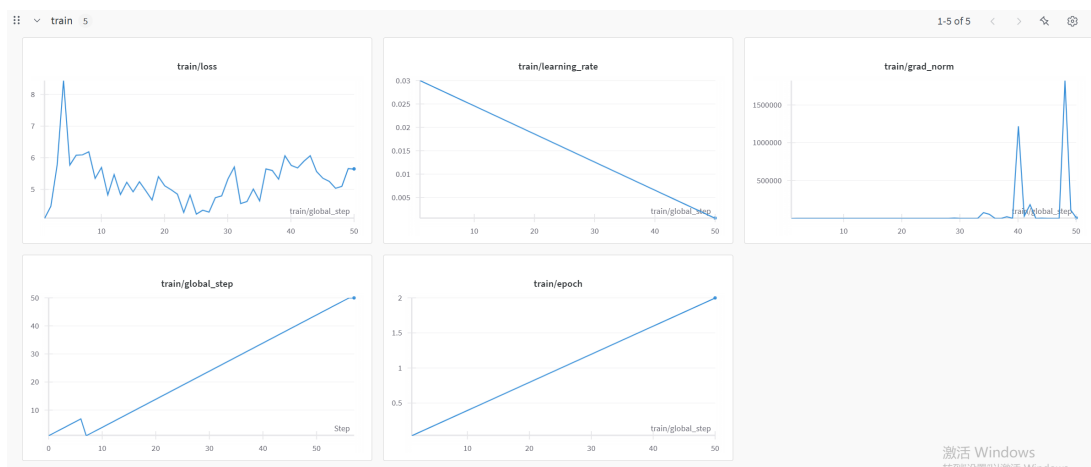


图 6: 基于 review 数据集 lora 微调的各种曲线

模型生成结果如下：

lora 微调前：[‘I love this movie because it is so funny and I am sure that my friends will enjoy too’]

lora 微调后：[“I love this movie because it is very good. I have never seen a film that made me feel like an idiot, and the only thing which was lacking in my opinion (and not even one of them) were some nice scenes with lots more action than anything else on screen).

The acting by John Huston has been great for years now but he still managed to make his mark as well at least when you saw him playing The Great Gatskills! It wasn’t just about being clever or stupid -”]

简单分析可以发现，微调后相比微调前，输出更加丰富，用词更加正式，英语表达也很少出现简单句，且更加流畅自然，我将微调前后的话给到 chatgpt-4o 来分析，给出的分析如下：【LoRA 微调后的模型在语言表达上更丰富、句式更复杂，展现出较强的生成能力，相较微调前简单重复输入的表现有明显提升。然而，微调后生成的内容出现了虚构信息和情绪偏离，可能偏离了原始任务目标，存在一定的语义不一致和“编故事”现象。这表明模型虽然学会了更自由的文本生成，但在任务对齐和输出可控性方面仍需优化，微调数据和目标需进一步明确与约束。】chatgpt 将我的感受以更加正式的文字表达出来。

3.3 调试 lora 超参数（开始炼丹!）

给定的代码是使用 peft（Parameter-Efficient Fine-Tuning）库配置了 LoRA 微调参数，具体参数设置如下：

首先加载Lora所需要用到的参数 Config

```
import peft
from peft import LoraConfig, get_peft_model, PeftModel

lora_config = LoraConfig(
    r=4, #As bigger the R bigger the parameters to train.
    lora_alpha=1, # a scaling factor that adjusts the magnitude of the weight matrix. Usually set to 1
    target_modules=["query_key_value"], #You can obtain a list of target modules in the URL above.
    lora_dropout=0.05, #Helps to avoid Overfitting.
    bias="lora_only", # this specifies if the bias parameter should be trained.
    task_type="CAUSAL_LM"
)
```

图 7: 修改 TrainingArguments

其中各项参数含义如下

- **r = 4**：表示低秩矩阵的秩，是 LoRA 注入模块的参数容量控制参数， r 越大，可训练参数越多。 $r = 4$ 是轻量配置，适合快速微调和资源受限场景（通常如 4、8、16 等）。
- **lora_alpha = 1**：缩放因子 α ，用于调节 LoRA 注入输出的强度，实际生效值为 $\frac{\alpha}{r}$ 。设置为 1 表示不进行额外缩放（通常与 r 成比例）。
- **target_modules = ["query_key_value"]**：指定对模型中的哪些模块注入 LoRA 权重，常见模块如 `q_proj`、`k_proj`、`v_proj` 等。此处选择 `query_key_value`，聚焦于 Transformer 的注意力层。
- **lora_dropout = 0.05**：对 LoRA 层引入 5% 的 Dropout，用于防止过拟合，尤其在小数据集微调中有良好效果。

- **bias = "lora_only"**: 仅对 LoRA 层中的偏置参数进行训练，是一种在训练效率与性能之间取得平衡的设置。
- **task_type = "CAUSAL_LM"**: 指定任务类型为自回归语言建模 (Causal Language Modeling)，适用于 GPT 类模型的下一个 token 预测任务。

colab 免费 gpu 额度用完,我决定在本地开始调参,路径如下:hw1-code/lora/Lora_based_on_review.ipynb, 在训练时, 出现如下问题:

Listing 2: lora 本地微调训练时报错

```
CommError: Run initialization has timed out after 90.0 sec. Please try increasing the timeout with the init_timeout setting: wandb.init(settings=wandb.Settings(init_timeout=120)).
```

核心问题在于本地电脑代码根本无法连接到 wandb 的服务器 (api.wandb.ai)

```
(llmba) D:\Desktop\Large-Language-Models-and-Alignment>curl https://api.wandb.ai
curl: (28) Failed to connect to api.wandb.ai port 443 after 21043 ms: Couldn't connect to server

(llmba) D:\Desktop\Large-Language-Models-and-Alignment>
```

图 8: 无法连接 wandb 服务器

暂时搁置, 优先处理后续问题

3.4 lora 微调时, 修改训练数据集为 Alpaca 数据集

将微调的数据集替换成 Alpaca 数据集, 原始代码有几处需要修改:

最明显一定是把数据集替换掉——dataset = "noob123/imdb_review_3000" 替换为 dataset = load_dataset("tatsu-lab/alpaca")

第二处是数据预处理, Alpaca 数据是 instruction + input → output 格式, 需拼接为 prompt:

Listing 3: 拼接为 prompt

```
def format_alpaca(example):
    instruction = example["instruction"]
    input_text = example.get("input", "")
    prompt = f"###_Instruction:\n{instruction}\n\n###_Input:\n{input_text}\n\n###_Response:\n"
    return {"prompt": prompt, "response": example["output"]}

formatted_dataset = dataset["train"].map(format_alpaca)
```

然后创建 tokenized 输入:

Listing 4: 创建 tokenized 输入

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bigscience/bloomz-560m")

def tokenize(example):
    input_ids = tokenizer(example["prompt"], truncation=True, padding="max_length", max_length=512).input_ids
```

```

labels = tokenizer(example["response"], truncation=True, padding="max_length", max_length
                    =512).input_ids
return {"input_ids": input_ids, "labels": labels}

tokenized_dataset = formatted_dataset.map(tokenize, remove_columns=formatted_dataset.
                                         column_names)

```

最后是在训练时，把 train_dataset 替换成 tokenized_dataset 即可，

完整代码见路径：hw1-code/lora/lora_based_on_Alpac.ipynb

在训练时出现很多问题，首先是我的参数设置导致训练时长很长，在 num_train_epochs=3 时要训练 6 个小时左右，num_train_epochs=1 要训练两个小时，其次的问题是训练时显存不足，是典型的 CUDA 内存不足 (OutOfMemoryError) 问题，报错信息如下：

Listing 5: CUDA 内存不足报错

```

OutOfMemoryError: CUDA out of memory. Tried to allocate 1.91 GiB. GPU 0 has a total capacity of
14.74 GiB of which 1.60 GiB is free. Process 4244 has 13.14 GiB memory in use. Of the
allocated memory 12.82 GiB is allocated by PyTorch, and 185.92 MiB is reserved by PyTorch
but unallocated. If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation. See documentation
for Memory Management (https://pytorch.org/docs/stable/notes/cuda.html#environment-
variables)

```

截至到显存不足时，训练情况如下所示：

首先加载Lora所需要用到的参数 Config

```

import peft
from peft import LoraConfig, get_peft_model, PeftModel

lora_config = LoraConfig(
    r=4, #As bigger the R bigger the parameters to train.
    lora_alpha=1, # a scaling factor that adjusts the magnitude of the weight matrix. Usually set to 1
    target_modules=["query_key_value"], #You can obtain a list of target modules in the URL above.
    lora_dropout=0.05, #Helps to avoid Overfitting.
    bias="lora_only", # this specifies if the bias parameter should be trained.
    task_type="CAUSAL_LM"
)

```

图 9: 基于 Alpaca 数据集 lora 微调的各种曲线

3.4.1 基于 Alpaca 数据集调试 lora 超参数（再次炼丹）

前述微调过程的问题暂时没有解决，这里首先从理论上分析二者调参的差别：在调试 LoRA 超参数时，IMDb Review 数据集因任务简单、数据量小，适合使用较小的 r（如 4）、较低的 alpha（如 16）和较高的 dropout（如 0.1）以防止过拟合；而 Alpaca 数据集属于复杂的指令生成任务，数据量大、输出多样，推荐使用更大的 r（如 8-16）、更高的 alpha（如 32-64）、较低的 dropout（如 0.05），并在更多注意力模块上应用 LoRA，以提升生成能力和上下文理解效果。总结为：两者的超参数差异源于任务性质、数据规模和模型微调目标的不同。